

RAG 知识库检索增强生成系统：工程设计文档

方向 B：智能客户支持与检索增强生成助手

1. 执行摘要

业务问题：企业员工每天在数百份非结构化文档中查找答案，传统关键词搜索无法理解'交作业'与'提交方式'的语义等价关系，导致重复提问率高、信息检索效率低下。

为什么传统方法做不到：关键词搜索依赖字面匹配——用户搜'怎么交作业'，文档中写的是'提交方式'，返回零结果。当文档规模超过数百份且表述方式多样时，基于 TF-IDF 或 BM25 的检索召回率急剧下降。

核心交付：我们构建了一个端到端 RAG 系统，从 176 篇杂乱文档自动构建可搜索向量索引。系统在 41 个域内查询上实现 87.8% 的 Recall@3，端到端稳态延迟 6.2 秒，建库成本 < 1。

2. 系统架构

系统采用七层流水线架构：ingest.py（摄取）→ clean_text()（清洗）→ chunk_text()（分块：段落→句子→贪心→滑动）→ extract_metadata()（LLM元数据）→ Qwen3-Embedding-0.6B（本地GPU，1024维）→ ChromaDB（HNSW, cosine）→ query_parser.py（意图解析）→ qa.py（答案生成）。

2.1 技术栈

层级	组件	技术选择
文档摄取	PDF/MD/TXT	PyMuPDF + PyYAML
文本清洗	正则流水线	Python re
语义分块	四层算法	自研（700字符，120重叠）
元数据提取	LLM结构化	DeepSeek V4 Flash（32线程）
向量嵌入	本地GPU推理	Qwen3-Embedding-0.6B（1024维）
向量数据库	HNSW索引	ChromaDB（cosine距离）
查询解析	LLM意图提取	DeepSeek V4 Flash
答案生成	LLM+来源约束	DeepSeek V4 Flash
前端	Web界面	Streamlit + 自定义CSS

2.2 模块职责

模块	职责
ingest.py	递归读取 .md/.txt/.pdf，解析 Front-Matter
collect_*.py	Wikipedia/SO/CSDN 三源 API 采集
preprocess.py	清洗+分块+元数据提取（431行）
embed_store.py	ChromaDB 封装：嵌入、检索、安全删除
query_parser.py	自然语言 → {search_query, filters}
qa.py	System Prompt 约束 + 来源强制标注
main.py / streamlit_app.py	CLI/Web 双入口

2.3 数据生命周期追踪

以一个具体查询为例，追踪数据在系统中的完整生命周期：

用户问'2025年的通知有哪些？'（1）query_parser.py 调用 LLM，输出 {search_query: '通知', filters: {year:2025, category:'notice'}}；（2）embed_store.py 对'通知'做向量嵌入（Qwen3，1024维，50ms）；（3）ChromaDB 在 481 个文档块上执行 HNSW 近似最近邻搜索，结合 where 过滤，返回 Top-3 结果；（4）qa.py 将检索到的 3 个块拼接为 ~2000 token 的上下文 System Prompt；（5）DeepSeek V4 Flash 生成带来源标注的答案。全过程端到端 6.2 秒，其中 LLM 调用占 96%。

3. 设计决策与取舍

3.1 向量数据库：ChromaDB vs Milvus vs Qdrant vs Pinecone

方案	优势	劣势	决策
ChromaDB	pip install 即用	不支持分布式	选择
Milvus	生产级分布式	需 Docker+etcd+MinIO	过度工程化
Qdrant	Rust 高性能	需独立服务进程	部署成本高
Pinecone	全托管	付费 SaaS	数据安全风险

选择 ChromaDB 的理由：课程场景无并发/分布式需求，优先降低部署成本。已做抽象封装，未来可平滑迁移。已知代价：复合 where 条件需显式 \$and 操作符（§4.3详述）。

3.2 分块策略：四层语义分块

我们选择了四层优先级语义分块算法（段落 → 句子 → 贪心合并 → 滑窗切割），而非简单固定长度切分。chunk_size=700，overlap=120。

方案	优势	劣势
固定长度（500字符）	实现快	破坏语义完整性
四层语义分块	尊重段落/句子边界	实现复杂度高
标题感知分块	利用文档结构	依赖格式规范化

700 字符选择依据：< 400 导致语义碎片化，> 1000 稀释向量聚焦度。700 约合 350 中文字，平衡精度与完整性。

3.3 嵌入模型：Qwen3-Embedding-0.6B

方案	维度	成本	延迟	中文支持
Qwen3-0.6B	1024	免费 (GPU)	~50ms/条	优秀
OpenAI ada-002	1536	\$0.0001/1K	~200ms/条	良好
MiniLM-L6-v2	384	免费	~30ms/条	一般

选择理由：(1)本地GPU推理，零API成本，建库 450+ 块嵌入费用 0；(2)Qwen3 原生中文语义理解优于英文为主的 MiniLM；(3)1024 维区分度优于 384 维。已知代价：首次加载 ~16s 冷启动。

3.4 LLM 选择：DeepSeek V4 Flash

方案	成本	延迟	质量
DeepSeek V4 Flash	~\$0.15/百万token	~2-4s	FAQ够用
GPT-4o-mini	\$0.15/百万token	~2-4s	质量稍优
本地LLM (7B)	免费	~10-30s (CPU)	不稳定

DeepSeek 与 GPT-4o-mini 同价位，但国内网络访问更稳定。本地 7B 模型在 CPU 上延迟 >10s，不适合交互场景。

3.5 查询解析：LLM 意图提取 vs 规则引擎

输入: "2024年的通知讲了啥"

输出: {search_query: "通知",
filters: {year:2024, category:"notice"}}

方案	优势	劣势
LLM 意图提取	理解自然语言，支持复合条件	额外 2-3s 延迟
规则引擎（正则）	零延迟	无法覆盖口语化表达
无解析（纯语义）	零开销	无法利用元数据过滤

回退策略：JSON 解析异常时，search_query 回退为原始问题，filters=None。评估中 3/50 个查询触发了此回退。

3.6 失败尝试：正则表达式分块 vs 语义分块

我们最初尝试用正则表达式 [。！？；] 将文档切为单句，直接按字符数累积到

chunk_size=700。这一方案在英文文档上效果良好，但在中文课程文档上出现严重问题：(1)

中文句中常含英文代码块，正则无法识别代码边界，代码被截断在块中间；(2) FAQ 短文本被切为两个独立块，语义关系丢失。结果：在 20 个中文 FAQ 查询中，正则方案 Recall@3 仅 62%，而四层语义方案达到 90%。代价：代码复杂度从 20 行增至 80 行，但召回率提升 28

个百分点。教训：纯正则忽略文档类型差异，递进式分块策略能对不同类型的文档自适应。

4. 评估与失效模式

4.1 评估方法

构建 50 个测试查询，覆盖 5 类：课程信息(10)、技术概念(10)、跨文档(10)、元数据过滤(10)、边界/超纲(10)。

4.2 评估结果

指标	数值	说明
Recall@3	87.8% (36/41)	域内查询 Top-3 检索命中率
幻觉率	1/9 (11.1%)	超纲查询中编造答案比例
平均延迟	5.6s (稳态 6.2s)	含首次冷启动
解析成功率	94% (47/50)	3次JSON解析失败自动回退

4.2.1 人工评估：答案相关性（1-5分）

我们对 50 个查询的生成答案进行了人工评分（5=完全准确，4=基本准确，3=部分相关，2=弱相关/误导，1=应拒答却编造）。

评分	数量	占比	典型场景
5 分	32	64%	技术概念全部满分；课程信息 Q3/Q5/Q8 满分
4 分	11	22%	跨文档轻微不完整；元数据过滤引用不精确
3 分	4	8%	检索到位但回答不精确（Q7/Q34/Q38/Q39）
2 分	2	4%	Q17 HNSW 未检索到；Q40 过滤空结果
1 分	1	2%	Q44 Python for 循环：超纲但 LLM 基于代码编造答案

域内查询平均评分 4.36/5。超纲拒答正确率 88.9% (8/9)。元数据过滤类评分最低 (3.5)，主要因 ChromaDB 复合 where 回退致过滤失效。

4.3 失效模式

失效 1：复合 where 条件不兼容。Q31 '2025年的通知'触发 ChromaDB 报错，回退为纯语义搜索。根因：ChromaDB 不支持隐式

AND。已修复：自动转为 \$and 格式。

失效 2：超纲查询幻觉。Q44 'Python for循环'检索到 SO 代码片段，LLM 给出详细代码——语义漂移：检索表面相关但主题不匹配。修复方向：System Prompt 增加主题相关性约束。

失效 3：查询解析 JSON 格式错误。Q32/Q38/Q39 返回非标准 JSON，正确回退但丢失过滤能力。修复方向：增加 JSON 修复逻辑。

4.4 事后剖析：两个重大失败

失败 1：SentenceTransformer 方法名变更。commit 92bc9b2 将 get_sentence_embedding_dimension() 改为 get_embedding_dimension()，基于 PyTorch deprecation 警告。但当前版本仍用旧方法名，导致全链路崩溃。单元测试因 Mock 了整个模型未发现。教训：Mock 测试无法发现 API 兼容性问题，需集成冒烟测试。

失败 2：极短文档静默丢弃。早期 clean_text() 对 <10 字符文档直接丢弃，导致 FAQ 短答案丢失。修复：chunk_text() 末尾兜底保留非空极短文档。教训：清洗应区分'噪声'和'短但有效'内容。

5. 延迟与成本估算

5.1 延迟预算

组件	首次查询	稳态查询	占比
模型加载(冷启动)	16.0s	0s	—
查询解析(LLM)	2.2s	2.7s	43%
向量检索	16.2s*	0.25s	4%
答案生成(LLM)	3.5s	3.2s	53%
端到端	22.0s	6.2s	100%

*首次检索含嵌入模型加载时间。瓶颈：LLM 调用占稳态延迟 96%，向量检索仅 250ms。

5.2 课程项目实际成本

调用类型	次数	单价	总成本
元数据提取(LLM)	176次	0.005	0.88
查询解析(LLM)	每次	0.005	按量
答案生成(LLM)	每次	0.01	按量
文本嵌入(本地)	~450次	0	0
建库总成本	—	—	< 1.0
单次查询成本	—	—	~ 0.015
每1000次查询	—	—	~ 15

5.2.1 每 1000 次查询成本详解

组件	单次	1000次	备注
查询解析(LLM)	0.005	5	~500 in + ~50 out tokens
答案生成(LLM)	0.01	10	~2000 ctx + ~500 out tokens
向量嵌入(本地)	0	0	Qwen3 GPU 免费推理
合计	0.015	15	

对比：若使用 OpenAI ada-002 做嵌入 + GPT-4o-mini 生成，每 1000 次约 50-80。本地嵌入节省 70% 查询成本。

5.3 云成本估算（10TB/天）

类别	月估算 ¥	依据
计算(ECS)	30,000-60,000	20×ecs.g6.4xlarge, 1.5/h
存储(OSS+ChromaDB)	10,000-20,000	10TB/天×30×0.12/GB
模型调用	20,000-80,000	LLM按查询量
合计	60,000-160,000	

附录 A：运行方式

```
pip install -r requirements.txt
copy .env.example .env # 填入 OPENAI_API_KEY
python src/main.py collect-all
python src/main.py build
python src/main.py ask --question "课程项目提交要求是什么?"
streamlit run app/streamlit_app.py
python -m pytest tests/ -v
```

附录 B：演示方案（15分钟）

环节	时长	内容
开场钩子	1min	业务问题引入
现场演示	3-4min	Streamlit 端到端问答
架构深潜	4min	数据生命周期追踪
我们搞砸了	2min	失败案例分享

Q&A

5min

技术总监提问